SIMATS ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

COURSE OUTCOME 3 (CO3) ASSIGNMENT

EVALUATE MEMORY-MANAGEMENT MECHANISMS: LINKING, PAGING, SEGMENTATION, DEMAND PAGING, PAGE REPLACEMENT AND DISK MANAGEMENT

Comprehensive Architectural Evaluation, Address Translation Workings, Belady's Anomaly Demonstration, Effective Access Time Derivations, and First-Principles Edge Memory Architecture

Student Name M.V. Rakesh Reddy
Register Number 19524164
Department Artificial Intelligence & Data Science (AI & DS)
Course & Code Operating Systems (CSA0413)
Course Faculty Kruthika K
Submission Date 04-11-2026
GitHub Repository Link `https://github.com/Rakeshreddymeruva/CSA0413-OS/tree/main/CO3`

Executive Summary & Progression

This evaluation report addresses Course Outcome 3 (CO3) for Operating Systems (CSA0413), rigorously executing the engineering lifecycle: **Design → Implementation → Analysis → Evaluation**. The analysis systematically examines memory management and non-volatile storage across real-world workloads:

- **Q1: Banking Application Deployment:** Evaluating static linking, dynamic linking, and runtime `dlopen()` loading for security hot-patching.
- **Q2: Multi-User College Server:** Overcoming contiguous memory external fragmentation via hardware paging; page table size derivations (32-bit vs. 48-bit); TLB hit ratio Effective Access Time (EAT) calculations.

- **Q3: Separate Code, Stack, and Data Segmentation:** Segment table limit translation, hardware boundary enforcement, segmentation fault traps, and Intel x86 segmented paging.
- **Q4: Medical Imaging Virtual Memory:** Executing a 24 GB imaging application on a 16 GB laptop via demand paging; page fault penalty bounds for 10% slowdown; thrashing and working-set modeling.
- **Q5: Web Server Page-Replacement:** Evaluating FIFO, Least Recently Used (LRU), and Optimal page replacement; mathematical proof and empirical demonstration of Belady's Anomaly on FIFO; Clock (second-chance) algorithm.
- **Q6: Hospital Records Storage:** Low-level disk formatting, partitioning, boot blocks, bad-block sparing, swap space sizing, and RAID 0, 1, 5, 6, 10 capacity/reliability models for 6 × 4 TB enterprise disks.
- **Q7: First-Principles Edge Memory Architecture:** Designing a deterministic memory-management plan for a smart factory edge device (2 GB RAM, no swap, zero page fault real-time control loop, 1.5 GB ML model) using `mlock()` and cgroups.

Q1 – Linking, Loading and Address Binding 6 Marks | K5

Problem Context: Updating a Banking Application

A bank operates hundreds of branch terminals. When a zero-day cryptographic vulnerability is patched in the core encryption module, distributing a newly compiled 200 MB monolithic executable across slow satellite links creates severe bandwidth strain, lengthy downtime, and version fragmentation.

Three Stages of Address Binding

1. **Compile-Time Binding:** If memory residency is known in advance, absolute machine code is generated. If the starting address shifts, the entire codebase must be recompiled (e.g., MS-DOS `.COM` binaries).
2. **Load-Time Binding:** The compiler outputs relocatable code. Symbol references are resolved to physical memory addresses by the loader when the program binary is initially loaded into RAM.
3. **Execution-Time (Dynamic) Binding:** Processes can be moved dynamically between memory segments during execution. Binding is deferred until instructions execute, relying on hardware Memory Management Unit (MMU) base and limit registers.

Roles of the Linker and the Loader

- **Linker (`ld`):** Combines compiled object files (`.o`) into a single executable, resolves external cross-module symbol references, arranges logical memory segments (text, data, bss), and creates dynamic linkage relocation tables (PLT/GOT).
- **Loader (`execve` / `ld.so`):** Reads the executable binary from storage, allocates user virtual address space, maps segments into memory pages, initializes the stack and heap, resolves shared library dependencies, and jumps to the program entry point (`_start`).

Static vs. Dynamic Linking & Runtime dlopen() Benchmark

Metric	Static Linking (`banking_app_static`)	Dynamic Linking (`banking_app_dynamic`)	Runtime Loading (`banking_app_dlopen`)
Binary Size on Disk	**762,472 bytes (~762 KB)**	**16,064 bytes (~16 KB) [47.5x smaller]**	16,128 bytes (~16 KB)
Physical RAM Sharing	Zero sharing; each process consumes duplicate code RAM.	**Single read-only copy** of `libencrypt.so` shared across all branch instances in physical RAM.	Loaded on demand; unmapped when finished via `dlclose()`.
Patching & Updatability	Requires redistributing full 200 MB binary to all branches.	**Instant hot-patching:** Replace `libencrypt.so` on disk; zero recompilation required.	Dynamic in-memory reload without restarting the main core service.
Execution Dependency	Self-contained; runs on any compatible kernel.	Fails immediately if `libencrypt.so` is missing at start.	Graceful degradation: Application catches `NULL` handle and alerts administrator.

Position-Independent Code (PIC) Mechanics

Shared libraries are compiled with `-fPIC` to allow loading at arbitrary physical and virtual addresses across different processes without code segment relocation. PIC implements two indirection structures:

- **Global Offset Table (GOT):** Located in the private data segment. Contains absolute physical pointers to global variables, updated by the dynamic linker at runtime.

- **Procedure Linkage Table (PLT):** Located in the shared code segment. Implements lazy symbol binding, resolving function addresses (e.g., `encrypt_data`) on their initial call.

Conclusion: Defended Linking Policy for the Bank

The bank must transition to **Dynamic Linking with Runtime `dlopen()` Fallback**. Dynamic linking shrinks executable sizes by 98%, permits instant deployment of critical security patches without recompiling branch binaries, and allows graceful error recovery if an encryption library is corrupted.

Q2 – Paging Architecture & Mathematical Derivations 8 Marks | K5

Problem Context: Multi-User College Server

A university compute server runs hundreds of student jobs of widely varying sizes. Contiguous allocation resulted in severe **External Fragmentation**: free memory was fragmented into thousands of small, non-contiguous holes, preventing 50 MB programs from loading despite 4 GB of aggregate free RAM.

Paging Principles: Eliminating External Fragmentation

Paging decouples logical memory from physical memory. Physical memory is divided into fixed-size frames (typically 4 KB), and logical address spaces are divided into pages of identical size. The operating system allocates any available free frame to any page.

- **External Fragmentation Elimination:** Because memory is allocated in uniform quanta, no unusable gaps exist between blocks. External fragmentation is **zero**.
- **Internal Fragmentation:** The final page allocated to a process is rarely filled completely. On average, a process wastes `PAGE_SIZE / 2` bytes on its terminal page. For 4 KB pages, average internal fragmentation is **2 KB per process**.

Mathematical Derivations

1. Page Table Size Calculations

- **32-Bit Logical Address Space with 4 KB Pages (2^{12 bytes):** Total Pages = rac{2^{32 bytes}{2^{12 bytes/page = 2^{20 = 1,048,576 pages Page Table Size = 2^{20 imes 4 bytes/entry = 4,194,304 bytes = 4 MB per process
- **48-Bit Virtual Address Space (x86_64) with 4 KB Pages (2^{12 bytes):** Total Pages = rac{2^{48}{2^{12 = 2^{36 = 68,719,476,736 pages Single-Level Table Size = 2^{36 imes 8 bytes/entry = 2^{39 bytes = 512 GB per process! *Architectural Solution:* This astronomical overhead necessitates a **4-Level Hierarchical Page Table** (PML4 ightarrow PDPT ightarrow PD ightarrow PT). Sparse memory spaces allocate intermediate table pages strictly on demand, reducing baseline overhead to ~64 KB per process.

2. Effective Access Time (EAT) with Translation Lookaside Buffer (TLB)

Given Memory Access Time t_m = 100 ns, TLB Search Time \epsilon = 0 ns. In standard paging, a TLB hit requires 1 memory access (100 ns); a TLB miss requires 2 memory accesses (1 to read page table + 1 to read physical memory = 200 ns): EAT = h imes 100 ns + (1 - h) imes 200 ns = 100 + 100(1 - h)

- **Hit Ratio h = 80\%:** EAT = 0.80(100) + 0.20(200) = 120 ns (20\% memory latency penalty).
- **Hit Ratio h = 95\%:** EAT = 0.95(100) + 0.05(200) = 105 ns (5\% penalty).
- **Hit Ratio h = 99\%:** EAT = 0.99(100) + 0.01(200) = 101 ns (Near-zero overhead).

Complete Code Implementation & Verification (`Q2/paging_simulation.c`)

```
#include <stdio.h>
#define PAGE_SIZE 1024
#define PAGES 8

/* Page table: frame number, or -1 if page is not resident in RAM */
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1;

int main(void) {
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049;
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];
```

```
        unsigned p = a / PAGE_SIZE;
        unsigned d = a % PAGE_SIZE;
        printf("%4u -> page %u offset %4u : ", a, p, d);
        if (p >= PAGES) {
            printf("invalid address\n");
         else if (page_table[p] == -1) {
            printf("page fault\n");
         else {
            printf("physical %u\n", page_table[p] * PAGE_SIZE + d);


    return 0;
```

Verified Linux Execution Output

./Q2/paging_simulation
0 -> page 0 offset 0 : physical 5120
1500 -> page 1 offset 476 : physical 2524
3100 -> page 3 offset 28 : physical 7196
6200 -> page 6 offset 56 : physical 3128
9000 -> page 8 offset 808 : invalid address
2049 -> page 2 offset 1 : page fault

Recalculation by Hand for PAGE_SIZE = 2048 Bytes

- **0:** p = 0/2048 = 0, d = 0. Frame = 5. Physical = 5 imes 2048 + 0 = 10240.
- **1500:** p = 1500/2048 = 0, d = 1500. Frame = 5. Physical = 5 imes 2048 + 1500 = 11740.
- **3100:** p = 3100/2048 = 1, d = 1052. Frame = 2. Physical = 2 imes 2048 + 1052 = 5148.
- **6200:** p = 6200/2048 = 3, d = 56. Frame = 7. Physical = 7 imes 2048 + 56 = 14392.
- **9000:** p = 9000/2048 = 4, d = 808. Frame = 0. Physical = 0 imes 2048 + 808 = 808.
- **2049:** p = 2049/2048 = 1, d = 1. Frame = 2. Physical = 2 imes 2048 + 1 = 4097.

Alternative Concept and Complexity Analysis: Inverted Page Table vs. Multi-Level Page Table

Space Complexity: A multi-level page table requires dedicated structures per process, scaling with total virtual address space O(Processes imes Virtual Space). An **Inverted Page Table (IPT)** maintains exactly one entry per *physical memory frame* across the entire machine, scaling strictly with physical RAM size O(Physical Frames). On a 64 GB server, an IPT requires only ~128 MB RAM, saving

gigabytes of memory.
Lookup Latency: Multi-level tables perform deterministic tree walks (O(L) where L = 4 memory references). IPTs require hashing logical addresses to locate frame indices. In the worst case (hash collisions), lookup degrades to linear scans O(Frames), requiring a dedicated hardware Inverted TLB to remain viable.

Q3 – Segmentation Architecture 6 Marks | K5

Problem Context: Separate Code, Stack, and Data

Programmers view software not as a flat sequence of bytes, but as distinct modular collections: read-only code shared by multiple compiler invocations, dynamically expanding stacks, variable-length heap allocations, and shared library routines. Segmentation directly matches this mental model.

Hardware Address Translation with Base and Limit

A logical address consists of a tuple: `<segment_number, offset>`. Address translation proceeds as follows:

1. The MMU indexes the Segment Table using `segment_number`.
2. The hardware compares `offset` against `limit`. If `offset ≥ limit`, the CPU immediately halts instruction execution and raises a privileged **Trap: Segmentation Fault (SIGSEGV)**.
3. If `offset < limit`, the hardware adds the base register to compute the physical memory address: Physical Address = base + offset

Verified Linux Execution Output (`Q3/segmentation_sim.c`)

./Q3/segmentation_sim
Logical Address (Seg 0, Offset 430) [Code] -> Physical Address: 649 (219 + 430)
Logical Address (Seg 1, Offset 10) [Data] -> Physical Address: 2310 (2300 + 10)
Logical Address (Seg 1, Offset 20) -> TRAP: Segmentation Fault (Offset 20 >= Limit 14)
Logical Address (Seg 2, Offset 50) [Stack] -> Physical Address: 140 (90 + 50)
Logical Address (Seg 3, Offset 400) [Heap] -> Physical Address: 1727 (1327 + 400)
Logical Address (Seg 4, Offset 95) [Shared]-> Physical Address: 2047 (1952 + 95)
Logical Address (Seg 4, Offset 100) -> TRAP: Segmentation Fault (Offset 100 >=

Limit 96)
Logical Address (Seg 5, Offset 10) -> TRAP: Invalid Segment

Architectural Comparison: Pure Paging vs. Pure Segmentation vs. Segmented Paging

Feature	Pure Paging	Pure Segmentation	Segmented Paging (Intel x86)
Programmer View	Linear, unsegmented byte stream.	**Modular** (Code, Data, Stack segments).	Modular segments divided into uniform pages.
Fragmentation	Zero external; internal exists (~2 KB/proc).	**Severe External Fragmentation** (Variable segments).	Zero external fragmentation; internal exists on final page.
Protection & Sharing	Complex (requires aligning segments to 4 KB page boundaries).	**Natural:** Code segment marked Read-Only/Execute.	Excellent: Segment registers enforce permissions; paging manages RAM.

Conclusion: Why Modern Operating Systems Rely Primarily on Paging

Modern 64-bit systems (Linux, Windows) rely primarily on pure paging because it completely eliminates external fragmentation and expensive memory compaction algorithms. Virtual memory protection bits (NX/XD execute disable, Write protect) in modern page tables deliver the security benefits of segmentation without its allocation complexity.

Q4 – Virtual Memory and Demand Paging 8 Marks | K5

Problem Context: Running 24 GB Medical Software on a 16 GB Laptop

A hospital radiologist must execute a large 3D medical volume rendering application requiring 24 GB of addressable memory on a diagnostic workstation equipped with only 16 GB of physical RAM.

Step-by-Step Hardware Page Fault Handling Lifecycle

1. **MMU Address Translation:** The radiologist's application references virtual address A. The MMU consults the page table and detects that the **Valid-Invalid bit** is set to `0` (Invalid / Not Present in RAM).
2. **Hardware Trap:** The MMU halts execution and triggers a hardware page fault exception (Ring 0 trap), pushing the faulting virtual address into the CPU register `%cr2`.
3. **Operating System Context Save:** The kernel saves process registers and verifies whether the memory access was a legitimate virtual address or an illegal access (which would emit a `SIGSEGV`).
4. **Locate Disk Block:** The kernel identifies the disk block backing the page in the swap area or binary file.
5. **Frame Allocation & Eviction:** The kernel allocates a free physical frame. If all 16 GB of RAM is full, the page replacement algorithm (e.g., LRU) selects a victim frame, writes it to disk if dirty, and updates its page table entry to invalid.
6. **Disk DMA Transfer:** The kernel programs the disk controller via DMA to read the requested 4 KB page from disk into the allocated physical frame. The faulting process is moved to the `WAITING` queue.
7. **Page Table Update:** Upon I/O completion, the kernel updates the page table entry with the new physical frame number, sets the valid bit to `1`, and marks the process as `READY`.
8. **Instruction Restart:** The CPU restores registers and re-executes the exact instruction that previously faulted.

Effective Access Time (EAT) Calculations

Given Memory Access Time t_m = 100 ns = 10^{-7 s, Page Fault Service Time t_f = 8 ms = 8 imes 10^{-3 s = 8,000,000 ns: EAT = (1 - p) imes t_m + p imes t_f = 100 + p(7,999,900) ns

- **Fault Rate p = 1/1,000 = 10^{-3:** EAT □pprox 100 + 10^{-3(8 imes 10^6) = 100 + 8,000 = 8,100 ns \quad (81 imes slower than RAM!)
- **Fault Rate p = 1/100,000 = 10^{-5:** EAT □pprox 100 + 10^{-5(8 imes 10^6) = 100 + 80 = 180 ns \quad (1.8 imes slowdown)
- **Fault Rate p = 1/1,000,000 = 10^{-6:** EAT □pprox 100 + 10^{-6(8 imes 10^6) = 100 + 8 = 108 ns \quad (8\% slowdown)

Maximum Acceptable Page Fault Rate for 10% Slowdown

Allowing a maximum 10% performance degradation (EAT \le 1.10 imes 100 ns = 110 ns): 100 + p(8 imes 10^6) \le 110 \implies p(8 imes 10^6) \le 10 \implies p \le 1.25 imes 10^{-6 The system must generate fewer than **1 page fault per 800,000 memory references** to prevent perceived lag.

Thrashing, Locality, and the Working-Set Model

- **Thrashing:** When a process's aggregate working set exceeds available physical memory frames (\sum WSS_i > D), the operating system spends >95% of its time swapping pages in and out of storage. Application throughput drops to near zero.
- **Copy-on-Write (COW):** When `fork()` is called, parent and child initially share identical physical memory pages marked Read-Only. Only when either process modifies a page does the MMU trap, prompting the kernel to allocate a new physical frame, clone the data, and mark it Read-Write. COW accelerates `fork()` by orders of magnitude.
- **Radiologist Conclusion:** While demand paging successfully allows a 24 GB app to run on 16 GB RAM, 3D volumetric rendering exhibits non-sequential spatial access across voxel arrays. **Upgrading to 32 GB RAM** is overwhelmingly superior to software locality tuning: eliminating page faults completely avoids the 8 ms disk latency penalty, guaranteeing smooth real-time diagnostic manipulation.

Q5 – Page-Replacement Techniques & Belady's Anomaly 8 Marks | K5

Problem Context: Web Server Page Caching

An enterprise web server caches active HTML pages in memory. Selecting an inefficient page-replacement policy triggers frequent secondary storage reads, inflating client response times.

Algorithms Evaluated

- **FIFO (First-In, First-Out):** Replaces the oldest page loaded in memory. Simple, but susceptible to replacing heavily used pages.

- **LRU (Least Recently Used):** Replaces the page that has not been accessed for the longest period. Requires hardware timestamps or stack tracking.
- **Optimal (OPT / MIN):** Replaces the page that will not be used for the longest period in the future. Theoretical upper bound (unimplementable in practice because future reference strings are unknown).
- **Clock (Second-Chance) Algorithm:** Practical O(1) approximation of LRU using a single reference bit per frame inspected by a rotating circular clock hand.

Complete Code Implementation (`Q5/page_replacement.c`)

```
#include <stdio.h>
#define MAXF 8

int find(const int f[], int n, int page) {
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
    return -1;


int lru(const int ref[], int len, int frames) {
    int f[MAXF], last[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) { last[k] = i; continue;
        faults++;
        // TODO 1 Completed: Allocate free frame or replace oldest last[]
        if (used < frames) {
            f[used] = ref[i]; last[used] = i; used++;
         else {
            int oldest = 0;
            for (int j = 1; j < frames; j++) {
                if (last[j] < last[oldest]) oldest = j;

            f[oldest] = ref[i]; last[oldest] = i;


    return faults;
```

Verified Linux Execution Output

./Q5/page_replacement
3 frames: FIFO 15, LRU 12, Optimal 9

4 frames: FIFO 10, LRU 8, Optimal 8
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults

Mathematical Explanation of Belady's Anomaly and the Stack Property

Formal Proof of Belady's Anomaly

Intuition suggests that allocating more physical memory frames must monotonically decrease or maintain the page fault count. **Belady's Anomaly** occurs when increasing frame allocation paradoxically *increases* page faults.

On the reference string `1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5`:

- With **3 Frames:** Generates exactly **9 page faults**.
- With **4 Frames:** Generates paradoxically **10 page faults**!

Root Cause (The Stack Property): Algorithms that satisfy the **Stack Property** guarantee that the set of pages resident in an N-frame system is always a strict subset of the pages resident in an (N+1)-frame system at all times t: M(N, t) \subseteq M(N+1, t) LRU and Optimal are stack algorithms; hence, they can *never* exhibit Belady's Anomaly. FIFO is not a stack algorithm: its eviction choice depends strictly on load arrival order, violating the subset invariant and discarding hot pages prematurely when an extra frame alters the queue alignment.

Q6 – Disk Management & RAID Architecture 4 Marks | K5

Problem Context: Storage for a Hospital Records Server

A central medical records server must boot reliably, guarantee continuous operations during physical disk failures, and maintain adequate virtual swap reserves for emergency patient admission spikes.

Disk Formatting & Bad-Block Management

- **Low-Level (Physical) Formatting:** Divides disk platters into hardware sectors with sector headers, ECC checksums, and inter-sector gaps. Performed by manufacturer.
- **Partitioning:** Slices raw storage into logical volumes (e.g., `/boot`, `/`, `/var/log`, `swap`).
- **Logical Formatting:** Writes operating system filesystem structures (inodes, superblocks, allocation bitmaps) onto partitions.
- **Boot Process:** System BIOS/UEFI executes POST, reads the Master Boot Record (MBR) or EFI System Partition, transfers control to the bootloader (GRUB), which loads the Linux kernel and `initramfs` into RAM.
- **Bad Block Sparing:** Modern disk controllers maintain a reserved pool of spare physical sectors. When a sector fails, the controller transparently re-maps the logical sector address to an intact spare sector (Sector Sparing / Forwarding).

RAID Comparison for 6 Disks of 4 TB (24 TB Raw Capacity)

RAID Level	Usable Capacity Formula	Usable Storage	Fault Tolerance	Read / Write Performance	Rebuild Risk
RAID 0	N imes C	**24 TB** (100%)	0 Disks (Zero resilience)	Max Read & Write	Total data loss if 1 disk fails.
RAID 1	(N / 2) imes C	**12 TB** (50%)	1 disk per mirror pair	High Read, Normal Write	Fast rebuild via mirror copy.
RAID 5	(N - 1) imes C	**20 TB** (83.3%)	Exactly 1 disk failure	High Read, 4-I/O Parity Write Penalty	High: Secondary disk failure during multi-hour rebuild causes total array loss.
RAID 6	(N - 2) imes C	**16 TB** (66.7%)	Any 2 disk failures	High Read, 6-I/O Double Parity Penalty	Resilient against dual disk failures.
RAID 10	(N / 2) imes C	**12 TB** (50%)	Up to 1 disk per stripe	**Maximum Read & Write**	**Fast rebuild, zero parity math.**

Hospital Records Recommendation: RAID 10 & Dedicated Swap Partition

RAID Level: We recommend **RAID 10**. Medical databases perform intensive random read/write transactions (vital chart updates, prescriptions). RAID 5 incurs

a severe 4-I/O parity update penalty and carries an unacceptable risk of unrecoverable read errors (URE) during multi-terabyte rebuilds. RAID 10 provides maximum transactional throughput and instantaneous rebuild.
Swap Architecture: Allocate a dedicated 16 GB partition on high-speed NVMe storage rather than a filesystem swap file to eliminate filesystem metadata traversal overhead during emergency swapping.

Q7 – Memory-Management Plan from First Principles 10 Marks | K5

Problem Context: Smart Factory Edge Device

An industrial edge gateway possesses **2 GB of physical RAM and no swap partition**. The device executes:

- A **Real-Time Control Task (200 MB):** Governs automated robotic safety valves. Must *never* be delayed by a page fault (strict zero-jitter bound).
- A **Machine-Learning Vision Task (1.5 GB):** Detects production assembly defects.
- A **System Logging Task (100 MB):** Collects telemetry and diagnostic metrics.
- *Future Growth:* A planned fourth task requiring **500 MB**.

Stage 1 – Interpret the Problem (2 Marks)

- **Real-Time Control:** Hard timing constraint (<1 ms cycle). Page fault tolerance = **ZERO**. A single 8 ms page fault triggers robotic mechanical collision.
- **Machine Learning:** Soft timing constraint (<500 ms per frame). Can tolerate occasional page faults, but requires 75% of physical memory.
- **Logging Task:** Best-effort background execution. Can tolerate significant page fault delays.
- **Measurable Engineering Target:** `Control Loop Latency < 500 µs at 99.999th percentile`; `System Free RAM ≥ 150 MB safety margin`.

Stage 2 – Compare Candidate Mechanisms (3 Marks)

Candidate Mechanism	Real-Time Task Suitability	Machine Learning Task Suitability	Memory Efficiency
Contiguous Allocation	Guarantees zero page faults, but suffers severe external fragmentation.	Cannot dynamically resize model weights.	Poor: Unusable memory gaps.
Pure Paging	Pages can be paged out under memory pressure, violating determinism.	Excellent: Easy frame allocation.	High: Zero external fragmentation.
Demand Paging with `mlockall()`	**Optimal:** Locks real-time code/data pages into physical RAM permanently.	**Optimal:** Pages model weights into remaining frames on demand.	**Maximum:** High utilization with absolute hard real-time safety.

Stage 3 – Implement and Verify (3 Marks)

The real-time control task executes `mlockall(MCL_CURRENT | MCL_FUTURE)` during initialization:

```
#include <sys/mman.h>

int init_realtime_task(void) {
    // Lock all current and future allocated pages into physical RAM
    if (mlockall(MCL_CURRENT | MCL_FUTURE) != 0) {
        perror("mlockall failed");
        return -1;

    // Pre-fault stack memory to prevent lazy page allocation faults
    char stack_prefault[64 * 1024];
    memset(stack_prefault, 0, sizeof(stack_prefault));
    return 0;
```

Verified Linux Execution Output (`Q7/realtime_mlock.c`)

./Q7/realtime_mlock
=== Real-Time Memory Locking Demonstration (mlock) ===

Measuring memory access WITHOUT mlock...
Unlocked loop execution time: 420.33 us
Locking memory frames via mlock()...
Memory frames locked into physical RAM successfully.
Locked loop execution time: 126.75 us (3.3x faster, deterministic bound)

Stage 4 – Defend the Design & 500 MB Task Expansion (2 Marks)

- **Handling the Fourth Task (500 MB):** Total memory demand = 200 MB (RT) + 1,500 MB (ML) + 100 MB (Log) + 500 MB (New) = 2,300 MB > 2,048 MB Physical RAM. Because the system lacks a swap disk, the Linux kernel Out-Of-Memory (OOM) killer would trigger and terminate processes indiscriminately.
- **Architectural Solution:**
  1. Protect the RT process: set `/proc/<pid>/oom_score_adj = -1000`, ensuring the kernel will never terminate the safety valve task.
  2. Configure **cgroups v2 (control groups)**: Cap the ML container at 1,200 MB and the logging container at 64 MB. Implement ZRAM (in-memory LZ4-compressed RAM swap) to dynamically compress inactive ML weight tensors by 3:1, releasing 400 MB of physical frames to comfortably accommodate the 500 MB new task.
- **Remaining Limitation:** While `mlock()` guarantees zero page-fault delays, it cannot protect against hardware cache line eviction and memory bus contention caused by heavy ML GPU/DMA transfers. Hardware core affinity (`taskset`) must isolate the RT task onto a dedicated CPU core.
